



# **Investigating the Limits of Tabular Self-Supervised Learning Across Dataset Sizes**

Prepared by:

Zain Qerba (160201)

Supervisor:

Dana El Rushaidat

January 2026

## نموذج حقوق الملكية الفكرية لمشاريع التخرج في قسم علوم الحاسوب

يتم قراءة وتوقيع هذا النموذج من قبل الطلاب المسجلين لمشاريع التخرج في قسم علوم الحاسوب

تعود حقوق الملكية الفكرية لمشاريع التخرج ونتائجها (مثل براءات الاختراع أو أي منتج قابل للتسويق) إلى جامعة العلوم والتكنولوجيا الأردنية، وتخضع هذه الحقوق إلى قوانين وأنظمة و تعليمات الجامعة المتعلقة بالملكية الفكرية وبراءات الاختراع.  
بناءً على ما سبق أوافق على ما يلي:

- (1) أن أحفظ كافة حقوق الملكية الفكرية لجامعة العلوم والتكنولوجيا الأردنية في مشروع التخرج.
- (2) أن ألتزم بوضع اسم جامعة العلوم والتكنولوجيا الأردنية و أسماء جميع الباحثين المشاركين في المشروع على أي نشرة علمية للمشروع كاملاً أو لنتائجه. و يشمل ذلك النشر في المجلات و المؤتمرات العلمية عامة أو النشر على المواقع الإلكترونية أو براءات الاختراع أو المسابقات العلمية.
- (3) أن ألتزم بأسس حقوق التأليف المعتمدة في جامعة العلوم والتكنولوجيا الأردنية.
- (4) أن أقوم بإعلام الجهة المختصة في الجامعة عن أي اختراع أو اكتشاف قد ينتج عن هذا المشروع و أن ألتزم السرية التامة في ذلك و أن أعمل من خلال الجامعة على الحصول على براءة الاختراع التي قد تنتج عن هذا المشروع.
- (5) أن تكون جامعة العلوم والتكنولوجيا الأردنية هي المالك لأي براءة اختراع قد تنتج عن هذا المشروع و تشمل هذه الملكية حق الجامعة في إعطاء التراخيص و التسويق و البيع كمؤسسة راعية و داعمة لكافة الأنشطة البحثية. ويكون حق للطالب شمول اسمه على براءة الاختراع كأحد المخترعين، و في حال تم إعطاء تراخيص أو تسويق و بيع لأي من منتجات المشروع يمنح المخترعون بما فيهم الطالب نسبة من الإيرادات حسب تعليمات البحث العلمي في جامعة العلوم والتكنولوجيا الأردنية.

مختص

للتوقيع

إسم الطالب ابراهيم حنف مربي

للتوقيع

إسم الطالب

للتوقيع

إسم الطالب

التوقيع

إسم المشرف

تاريخ .....

# Investigating the Limits of Tabular Self-Supervised Learning Across Dataset Sizes

Zain Qerba

Department of Computer Science  
Jordan University of Science and Technology  
Irbid, Jordan  
ziqerba22@cit.just.edu.jo

Dana El Rushaidat

Department of Computer Science  
Jordan University of Science and Technology  
Irbid, Jordan  
dmelrushaidat@just.edu.jo

**Abstract**—Tabular data is widely used in real-world machine learning applications, yet it remains unclear when self-supervised learning (SSL) provides a practical advantage over strong tree-based baselines. This paper investigates the labeled-data crossover point at which SSL-based or pretrained deep tabular models begin to outperform an Optuna-tuned XGBoost baseline. Four deep learning configurations are evaluated across eight labeled-data fractions ranging from 100% to 1%: TabNet on Forest Cover Type, SAINT on Adult Income, TabPFN v2 on California Housing, and a T-JEPA→TabPFN pipeline.

Results show that no universal crossover point exists across datasets. XGBoost remains highly competitive, outperforming TabNet on Covertypes across all fractions by accuracy, while TabNet’s reconstruction-based SSL was unstable at the full dataset scale. On Adult Income, SAINT only surpasses XGBoost in the extreme 1% labeled-data regime, achieving a ROC-AUC of 0.8596 compared with 0.8571 for XGBoost. On California Housing, TabPFN v2 outperforms XGBoost at all labeled-data fractions, achieving the strongest regression results overall. However, the proposed T-JEPA→TabPFN pipeline does not improve over raw TabPFN, suggesting overlap between TabPFN’s in-context learning and external SSL representation learning. Overall, the results suggest that SSL is most useful only in extreme low-label settings, while pretrained tabular foundation models may offer a stronger direction for improving tabular learning.

## I. PROJECT GOALS AND OBJECTIVES

**Establish a strong tabular baseline:** Train XGBoost with Bayesian hyperparameter optimization using Optuna across three benchmark datasets and eight labeled-data fractions from 100% to 1%, ensuring that deep learning comparisons are made against a competitive tree-based baseline rather than an under-tuned model.

**Estimate labeled-data crossover points:** Identify whether a labeled-data crossover point exists for each model-dataset pair, defined as the smallest training fraction at which an SSL-enhanced or pretrained deep model outperforms XGBoost, and analyze whether this behavior is consistent across classification and regression tasks.

**Evaluate SSL-enhanced deep learning models:** Benchmark TabNet with masked feature reconstruction on Forest Cover Type and a custom SAINT implementation combining denoising and contrastive SSL on Adult Income, assessing whether dataset-specific SSL pretraining improves performance as labeled data decreases.

**Evaluate a tabular foundation model:** Assess TabPFN v2 on California Housing as a pretrained foundation model that performs in-context prediction without conventional dataset-specific gradient training, testing whether large-scale synthetic pretraining provides a more consistent advantage than target-dataset SSL.

**Introduce a novel SSL-to-foundation-model pipeline:** Design and implement T-JEPA→TabPFN, combining T-JEPA self-supervised representation learning with TabPFN in-context learning, a combination not previously studied, and evaluate whether SSL-learned embeddings improve foundation model predictions.

**Analyze positive and negative findings:** Document both successful and unsuccessful outcomes, including SSL pretraining instability and cases where SSL provides no performance benefit, treating these results as empirical evidence to guide future tabular SSL research.

## II. INTRODUCTION

Tabular data is one of the most common data formats in real-world machine learning applications, including finance, healthcare, logistics, and scientific analysis. Despite the success of deep learning in image, speech, and language tasks, tabular prediction remains a challenging setting for neural networks. Unlike images, which contain spatial structure, or text, which follows a sequential order, tabular data often consists of heterogeneous columns with different meanings, scales, and data types. Continuous, categorical, binary, and ordinal variables may appear together, and the position of a column usually does not carry the same structural meaning as a pixel location or word position.

These characteristics make tabular data difficult for deep learning models. Neural networks often require strong inductive biases, careful preprocessing, regularization, and hyperparameter tuning to perform well. In contrast, decision-tree-based models, especially gradient-boosted trees such as XGBoost, remain highly competitive in practical tabular prediction tasks [1]–[3]. They naturally handle nonlinear feature interactions, mixed feature scales, and irregular feature distributions without requiring learned feature embeddings. For this reason, any proposed deep learning improvement for tabular

data should be compared against a carefully tuned tree-based baseline.

Self-supervised learning (SSL) has been proposed as one way to improve deep learning for tabular data, especially when labeled examples are scarce [4]–[7]. Instead of relying only on target labels, SSL learns representations from unlabeled data through pretext tasks such as reconstructing masked features, denoising corrupted inputs, contrasting different views of the same sample, or predicting latent feature representations. In principle, this allows a model to learn useful relationships between features before supervised training begins. However, SSL for tabular data is less straightforward than SSL for images or text. Meaningful augmentations are harder to define, feature corruption can create unrealistic samples, and reconstruction objectives may become unstable when features have very different scales or distributions.

Although prior work has reported promising supervised and SSL-based results for several deep tabular models, unsuccessful pretraining attempts and dataset-specific SSL limitations are not always fully visible from published benchmark tables. For example, TabNet reports strong supervised performance on the Covertype dataset, while its self-supervised pretraining results are mainly demonstrated in other settings [8]. This leaves open whether masked feature reconstruction is stable and useful at Covertype scale. Reporting such negative or unstable outcomes is important because it can prevent future researchers from repeating costly experiments whose limitations are not clearly documented.

This creates an important practical question: under what labeled-data conditions does SSL actually improve tabular deep learning compared with a strong XGBoost baseline? While SSL is often motivated by label scarcity, its benefits are not guaranteed across datasets, model architectures, or task types. In some settings, the cost of pretraining may not be justified if a tuned tree-based model remains stronger. In other settings, pretrained foundation-style tabular models may offer a more reliable advantage than dataset-specific SSL.

This paper investigates this question through a controlled dataset-scaling experiment. An Optuna-tuned XGBoost baseline is compared against four deep learning configurations across eight labeled-data fractions: 100%, 50%, 10%, 8%, 6%, 4%, 2%, and 1%. Three benchmark datasets are used: Forest Cover Type for multiclass classification, Adult Income for imbalanced binary classification, and California Housing for regression [9]–[11]. TabNet with masked feature reconstruction is evaluated on Covertype, while a custom SAINT implementation using denoising and contrastive SSL is evaluated on Adult Income [7], [8]. TabPFN v2 is evaluated on California Housing as a pretrained tabular foundation model, not as a target-dataset SSL model [12], [13]. It is pretrained offline on synthetic tabular tasks and performs in-context prediction without conventional dataset-specific gradient training. In addition, this work explores a T-JEPA→TabPFN pipeline, where T-JEPA is used to learn self-supervised tabular representations before passing the learned embeddings to TabPFN [14]. To the best of our knowledge, this specific SSL-to-foundation-

model combination has not been previously evaluated in the published literature.

The findings show that there is no universal crossover point where SSL-based deep learning consistently overtakes XGBoost across all datasets. On Covertype, XGBoost remains stronger in accuracy across all labeled-data fractions, while TabNet achieves a virtually identical Macro-F1 at the full-data setting (0.9390 vs. 0.9389), and XGBoost leads at all reduced-data fractions. TabNet’s reconstruction-based SSL was also unstable at this dataset scale, suggesting that the absence of reported SSL results for Covertype in prior TabNet work is practically important. On Adult Income, SAINT surpasses XGBoost only in the extreme 1% labeled-data setting, achieving a ROC-AUC of 0.8596 compared with 0.8571 for XGBoost. On California Housing, TabPFN v2 provides the strongest regression performance overall, outperforming XGBoost at every fraction, including at the full-data setting where it achieves lower RMSE than XGBoost (0.3905 vs. 0.4424). However, the proposed T-JEPA→TabPFN pipeline does not improve over raw TabPFN, indicating that external SSL representations and TabPFN’s in-context learning may capture overlapping feature information. Overall, the results suggest that dataset-specific SSL is not a guaranteed replacement for tuned tree-based models, while pretrained tabular foundation models may provide a more reliable direction for robust tabular learning.

### III. RELATED WORK

#### A. Tree-Based Baselines and Tabular Deep Learning

Tree-based ensemble models remain among the strongest baselines for tabular prediction tasks. Methods such as XGBoost, LightGBM, and CatBoost are widely used because they naturally handle nonlinear feature interactions, mixed feature scales, sparse features, and irregular decision boundaries [1], [15], [16]. These properties make them highly competitive on small- and medium-sized tabular datasets, where neural networks may struggle to learn robust feature representations from limited labeled data [2], [3].

This does not mean that tree-based models always dominate every low-label setting. Rather, their inductive bias makes them difficult to outperform unless the deep model can exploit additional information, such as unlabeled data through self-supervised learning or prior knowledge from large-scale pretraining. This distinction is important for the present study. Instead of assuming that SSL-based deep models are better under label scarcity, this work evaluates whether they actually outperform a strong Optuna-tuned XGBoost baseline across progressively smaller labeled-data fractions.

#### B. Self-Supervised Learning for Tabular Data

Self-supervised learning has been proposed as a way to improve deep tabular models when labeled data is limited. Instead of relying only on target labels, SSL learns representations from unlabeled data using pretext tasks. In tabular learning, common SSL strategies include predictive learning,

contrastive learning, hybrid learning, and representation-space learning [4]–[6], [17].

Predictive SSL methods mask, corrupt, or remove parts of the input and train the model to reconstruct the missing information. This encourages the model to learn dependencies between features. However, this objective can be unstable for tabular data because columns often have different scales, distributions, and semantic meanings. If large-scale numerical features dominate the reconstruction loss, the model may fail to learn useful representations.

Contrastive SSL methods create multiple views of the same sample and train the model to align related views while separating unrelated samples. This approach is powerful in domains such as images, where augmentations like cropping or rotation usually preserve semantic meaning. In tabular data, however, defining meaningful augmentations is harder. Random masking, feature replacement, CutMix, mixup, or noise injection may create unrealistic records or remove important feature relationships. Hybrid SSL methods combine reconstruction and contrastive objectives to capture both feature dependency and representation invariance.

Representation-space SSL methods attempt to avoid some of the weaknesses of input-space corruption by applying transformations after the raw input has already been encoded. For example, representation-space augmentation methods such as RaTab shift augmentation from raw features into latent representations, aiming to preserve important tabular structure while still creating useful contrastive views [17]. This line of work supports the broader motivation that tabular SSL should avoid unrealistic input corruptions when possible.

The difficulty of designing reliable SSL objectives motivates the main question of this work: whether SSL pretraining actually provides a practical advantage over a tuned XGBoost baseline as labeled data decreases.

### C. TabNet and Masked Feature Reconstruction

TabNet is a deep tabular architecture that uses sequential attention to perform instance-wise feature selection [8]. At each decision step, the model learns which features to focus on, allowing it to combine prediction with interpretability. This sparse feature-selection mechanism is designed to make TabNet more suitable for tabular data than generic multilayer perceptrons.

TabNet also includes a self-supervised pretraining procedure based on masked feature reconstruction. In this setup, parts of the input are masked, and the model learns to reconstruct the missing values before supervised fine-tuning. The original TabNet work reports strong supervised performance on several datasets, including Covertype, and demonstrates unsupervised pretraining benefits mainly on other settings such as Higgs [8].

This leaves an important gap for Covertype. Although TabNet reports strong supervised performance on Covertype, the paper does not clearly establish whether masked feature reconstruction is stable and useful on Covertype at full training scale. This work directly tests that missing setting by applying TabNet’s SSL pretraining to Covertype across

multiple labeled-data fractions. The negative result is important: TabNet’s reconstruction-based pretraining was unstable at this scale, meaning that the final model behaved mostly like a supervised TabNet model rather than a successful SSL-enhanced model.

### D. SAINT and Hybrid Tabular SSL

SAINT is a transformer-based architecture designed for tabular learning [7]. Its main contribution is the combination of attention across features within a sample and attention across samples within a batch. The original SAINT paper studies three architectural variants: SAINT-s, which uses self-attention only; SAINT-i, which uses intersample attention only; and SAINT, which combines both mechanisms. This distinction matters because self-attention captures relationships among features in the same row, while intersample attention allows a row to use information from other rows in the batch.

SAINT also studies self-supervised pretraining for tabular data. Its pretraining pipeline combines contrastive learning and denoising. The contrastive objective aligns two augmented views of the same sample, while the denoising objective trains the model to recover corrupted information. The original paper reports that the combination of contrastive and denoising losses performs better than using either loss alone, and that using CutMix together with mixup gives the strongest pretraining configuration [7], [18], [19].

In this work, SAINT is evaluated on Adult Income because it is a standard binary classification benchmark with mixed categorical and numerical features and class imbalance. Adult Income is also relevant to SAINT-style evaluation because it appears in prior SAINT benchmark settings. The implementation used in this project is a compact SAINT-style model built from scratch with row-level intersample attention, denoising reconstruction, and NT-Xent contrastive pretraining. Unlike the original SAINT pretraining setup, which uses CutMix and mixup augmentations, this implementation generates contrastive views by adding Gaussian noise to the input features. This choice keeps the implementation simple, reproducible, and independent of the original SAINT codebase.

Because this implementation first projects each row into a single dense embedding, it should be treated as a simplified SAINT-style implementation rather than a full reproduction of the original SAINT architecture. The purpose is to test whether hybrid SSL ideas from SAINT provide a measurable advantage over tuned XGBoost as labeled data becomes scarce.

### E. Joint-Embedding Predictive SSL: T-JEPA

Many tabular SSL methods rely on corrupting raw input features and reconstructing the original values. This can be problematic when feature corruptions produce unrealistic samples or when reconstruction loss is dominated by features with large numerical scales. T-JEPA addresses this issue using a joint-embedding predictive architecture for tabular data [14]. Instead of reconstructing raw feature values, T-JEPA predicts latent representations of masked feature subsets.

T-JEPA uses three main components: a context encoder, a target encoder, and a predictor. The context encoder processes the visible features. The target encoder processes the target feature subset and is updated using an exponential moving average of the context encoder. The predictor then uses the context representation to predict the target latent representation. This design moves the prediction task from raw input space to latent representation space, reducing the need for manually designed tabular augmentations. T-JEPA also introduces a regularization token to reduce representation collapse during training.

Prior T-JEPA experiments evaluate its learned representations with downstream tabular backbones such as MLP, ResNet, DCNv2, AutoInt, and FT-Transformer. These experiments show that T-JEPA can improve several deep tabular models and sometimes match or outperform gradient-boosted decision trees. However, prior work does not evaluate T-JEPA as a representation front-end for TabPFN. This paper therefore explores a T-JEPA→TabPFN pipeline in which T-JEPA learns representations from unlabeled California Housing features and TabPFNRegressor uses the resulting embeddings for downstream regression.

Since T-JEPA produces learned sample embeddings that can be represented as a standard tabular feature matrix after pooling, it is natural to ask whether replacing raw input features with these embeddings can improve TabPFN’s in-context learning.

#### *F. Tabular Foundation Models and TabPFN*

Recent work has introduced tabular foundation models as an alternative to training a new model from scratch for every dataset. TabPFN is a prominent example of this direction [12], [13]. Unlike TabNet, SAINT, and T-JEPA, TabPFN is not a target-dataset SSL model. It is pretrained offline on synthetic tabular tasks and then performs prediction through in-context learning. During inference, the available labeled examples are used as context, and the model produces predictions without conventional dataset-specific gradient training.

The original TabPFN work focused on classification, while TabPFN v2 provides both classification and regression interfaces. Therefore, this paper evaluates TabPFN v2 through TabPFNRegressor on the California Housing regression task. This allows TabPFN to serve as a pretrained foundation-model reference against both XGBoost and the proposed T-JEPA→TabPFN pipeline.

Real-TabPFN represents a related but separate direction. It improves TabPFNV2 through continued pretraining on curated real-world datasets, bridging synthetic pretraining and real-data pretraining [13]. However, Real-TabPFN is discussed only as related work in this paper; it is not the evaluated regression model. The experiments reported here use TabPFN v2 through TabPFNRegressor.

#### *G. Research Gap*

Prior work has introduced strong tabular deep architectures, SSL objectives, and foundation-model approaches. However,

several gaps remain. First, many studies report results at fixed dataset sizes rather than systematically varying the amount of labeled data. This makes it difficult to identify the labeled-data fraction at which SSL begins to outperform a tuned tree-based baseline. Second, negative SSL outcomes, such as unstable pretraining or cases where SSL provides no benefit, are not always emphasized in benchmark papers. Third, foundation models such as TabPFN are often discussed separately from target-dataset SSL methods, even though both aim to reduce dependence on labeled data.

This paper addresses these gaps through a controlled comparison of an Optuna-tuned XGBoost baseline, dataset-specific SSL models, a tabular foundation model, and an SSL-to-foundation-model pipeline across eight labeled-data fractions. The goal is not only to identify where deep learning outperforms XGBoost, but also to document when SSL fails to justify its additional complexity. Table I summarizes the representative prior results that motivate the experimental comparisons in this study.

### IV. METHODOLOGY

#### *A. Research Design Overview*

This study follows a controlled experimental design comparing a supervised tree-based baseline, XGBoost, against four deep learning configurations across three benchmark tabular datasets and eight labeled-data fractions. The goal is to identify whether self-supervised or pretrained deep tabular models begin to outperform XGBoost as labeled training data decreases.

The evaluated configurations are XGBoost, TabNet on Covertype, a simplified SAINT-style model on Adult Income, TabPFN v2 on California Housing, and a T-JEPA→TabPFN pipeline on California Housing. The labeled-data fractions are:

$$F = \{100\%, 50\%, 10\%, 8\%, 6\%, 4\%, 2\%, 1\%\}.$$

For each dataset, a fixed 80/20 train-test split is created once. The same test split is kept fixed across all fractions so that every model is evaluated on the same held-out samples. For classification datasets, stratified splitting is used to preserve class proportions. For regression, random splitting is used. The random seed is fixed at 42 for reproducibility across data splitting, subsampling, and model initialization where applicable.

At each labeled-data fraction, a subset of the training split is selected and used for supervised training or fine-tuning. For classification datasets, fraction subsampling is stratified to preserve class balance. For regression, random subsampling is used. Self-supervised pretraining is applied only to the relevant models and does not use target labels. TabPFN v2 is not trained through target-dataset SSL; it is used as a pretrained tabular foundation model.

#### *B. Datasets*

Three benchmark datasets are used to cover multiclass classification, imbalanced binary classification, and regression.

TABLE I  
REPRESENTATIVE PRIOR RESULTS FROM RELATED WORK

| Paper / Method          | Main Idea                                                                                                                            | Evaluation Scope in Original Work                                                                                     | Representative Reported Result                                                                                                                                                              | Relevance to This Study                                                                                                                                    |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>TabNet</b> [8]       | Sequential attention for instance-wise feature selection; optional self-supervised pretraining through masked feature reconstruction | Classification and regression datasets, including Covertypes and Sarcos; SSL pretraining demonstrated mainly on Higgs | Reports 96.99% test accuracy on Covertypes in a supervised setting; reports 0.14 MSE on Sarcos; SSL pretraining on Higgs improves accuracy from 57.47% to 61.37% with 1,000 labeled samples | Motivates testing whether TabNet’s masked reconstruction SSL is stable and useful on Covertypes, where the reported Covertypes result is supervised        |
| <b>SAINT</b> [7]        | Transformer-based tabular model with self-attention, intersample attention, and contrastive + denoising pretraining                  | Multiple tabular classification datasets and semi-supervised settings with limited labels                             | In the semi-supervised setting with 50 labeled samples, SAINT + pre-training reports 86.91 average AUROC, compared with 85.78 for supervised SAINT and 78.13 for XGBoost                    | Motivates testing whether SAINT-style SSL can outperform XGBoost under severe label scarcity                                                               |
| <b>T-JEPA</b> [14]      | Augmentation-free SSL that predicts masked feature representations in latent space                                                   | Evaluated as a representation learner with downstream models such as MLP, ResNet, DCNV2, AutoInt, and FT-Transformer  | On California Housing, DCNV2 + T-JEPA reports 0.420 RMSE, compared with 0.504 RMSE for DCNV2 without T-JEPA                                                                                 | Motivates testing whether T-JEPA embeddings can also improve TabPFNRegressor                                                                               |
| <b>TabPFN v2</b> [13]   | Tabular foundation model pretrained on synthetic tabular tasks and used through in-context prediction                                | Small tabular benchmark datasets; prediction without conventional dataset-specific gradient training                  | In the Real-TabPFN comparison, default TabPFNV2 reports a mean normalized ROC-AUC of 0.954 across 29 OpenML AutoML classification datasets                                                  | Motivates evaluating TabPFN v2 as a non-SSL foundation-model reference                                                                                     |
| <b>Real-TabPFN</b> [13] | Continued pretraining of TabPFNV2 on curated real-world datasets                                                                     | 29 OpenML AutoML Benchmark classification datasets                                                                    | Improves mean normalized ROC-AUC from 0.954 to 0.976 compared with default TabPFNV2                                                                                                         | Shows the direction of continued real-data pretraining beyond synthetic pretraining; not evaluated in this study — TabPFN v2 base weights are used instead |

**Forest Cover Type (Covertypes)** is a multiclass classification dataset containing 581,012 samples and 54 cartographic features. The task is to predict one of seven forest cover type classes. The dataset is strongly imbalanced, making both accuracy and Macro-F1 useful for evaluation. Covertypes is used for the TabNet experiment because TabNet reports strong supervised performance on this benchmark [8], [9].

**Adult Income** is an imbalanced binary classification dataset containing 48,842 samples and 14 features. The task is to predict whether annual income exceeds \$50,000. The dataset contains both numerical and categorical features and has an approximate 75/25 class imbalance. Adult Income is used for the SAINT-style experiment because it is a standard tabular benchmark and is suitable for testing SSL under class imbalance and label scarcity [7], [10].

**California Housing** is a regression dataset containing 20,640 samples and 8 numerical features. The target is median house value in units of \$100,000. This dataset is used for the TabPFN v2 and T-JEPA→TabPFN experiments because it is a standard tabular regression benchmark and appears in T-JEPA-related evaluation [11], [14].

### C. Data Splitting, Subsampling, and Preprocessing

Each dataset is split once into 80% training and 20% testing. Table II reports the resulting training and test split sizes for the three datasets.

Table III shows the selected training subset sizes used at each labeled-data fraction.

Preprocessing differs by model because each method has different input requirements.

TABLE II  
DATASET SPLIT SIZES

| Dataset            | Training Samples | Test Samples |
|--------------------|------------------|--------------|
| Covertypes         | 464,809          | 116,203      |
| Adult Income       | 39,073           | 9,769        |
| California Housing | 16,512           | 4,128        |

TABLE III  
TRAINING SUBSET SIZES BY LABELED FRACTION

| Fraction | Covertypes | Adult Income | California Housing |
|----------|------------|--------------|--------------------|
| 100%     | 464,809    | 39,073       | 16,512             |
| 50%      | 232,404    | 19,536       | 8,256              |
| 10%      | 46,480     | 3,907        | 1,651              |
| 8%       | 37,184     | 3,125        | 1,320              |
| 6%       | 27,888     | 2,344        | 990                |
| 4%       | 18,592     | 1,562        | 660                |
| 2%       | 9,296      | 781          | 330                |
| 1%       | 4,648      | 390          | 165                |

For Covertypes, labels are shifted from 1–7 to 0–6 for compatibility with Python model implementations. In the XGBoost baseline, StandardScaler is fitted on the training split and applied to the test split. In the TabNet experiment, StandardScaler is fitted separately on the selected training fraction and applied to the fixed test split.

For Adult Income, raw labels are cleaned by stripping whitespace and trailing dots before encoding. Categorical

features are label-encoded before the train-test split. The preprocessing code also includes NaN-filling safeguards before the split, although the notebook does not record a separate missing-value count. After the split, StandardScaler is fitted on the training split and applied to the test split. This preprocessing is retained as implemented, while recognizing that a stricter future replication would fit categorical encoders using only the training split.

For California Housing, XGBoost uses StandardScaler fitted on the training split and applied to the test split. Raw TabPFN v2 is applied without external scaling because TabPFN performs internal preprocessing. For the T-JEPA→TabPFN pipeline, StandardScaler is fitted on the training features before T-JEPA pretraining and applied to the test features. After T-JEPA embedding extraction, a second StandardScaler is fitted on the training embeddings and applied to the test embeddings before TabPFNRegressor inference.

#### D. Dataset Scaling Protocol

To study how model performance changes as labeled data decreases, each model is evaluated across the eight labeled-data fractions listed in Section IV-A. Subsampling is applied only to the training split. The test split remains fixed and identical across all fractions for a given dataset.

For classification datasets, stratified subsampling is used to preserve class proportions at every fraction. This is especially important at the 1% fraction, where random sampling could underrepresent rare classes. For regression, random subsampling is used because the target variable is continuous.

Additional intermediate fractions below 10% are included to provide better resolution in the low-label region, where model behavior is expected to change most sharply.

The crossover point for a model-dataset pair is defined as the smallest labeled-data fraction at which the deep learning configuration performs at least as well as XGBoost under the primary metric for that dataset. The primary metrics are accuracy for Covertypes, ROC-AUC for Adult Income, and RMSE for California Housing. For accuracy and ROC-AUC, higher values are better. For RMSE, lower values are better. If no fraction satisfies this condition, the model is treated as having no crossover point against XGBoost on that dataset.

#### E. Evaluation Metrics

The metrics are selected according to the task type.

For Covertypes, accuracy and Macro-F1 are used. Accuracy is defined as:

$$\text{Accuracy} = \frac{1}{n} \sum_{i=1}^n \mathbf{1}(y_i = \hat{y}_i),$$

where  $y_i$  is the true label,  $\hat{y}_i$  is the predicted label, and  $n$  is the number of test samples.

For class  $c$ , precision, recall, and F1 are defined as:

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c},$$

$$\text{Recall}_c = \frac{TP_c}{TP_c + FN_c},$$

$$F1_c = \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}.$$

Macro-F1 averages the class-wise F1 scores equally across all  $C$  classes:

$$\text{Macro-F1} = \frac{1}{C} \sum_{c=1}^C F1_c.$$

For Adult Income, ROC-AUC, F1, and PR-AUC are used. ROC-AUC measures the ranking quality of positive-class prediction scores across decision thresholds. It can be interpreted as the probability that a randomly selected positive sample receives a higher score than a randomly selected negative sample:

$$\text{ROC-AUC} = \frac{1}{n_+ n_-} \sum_{i: y_i=1} \sum_{j: y_j=0} \mathbf{1}(s_i > s_j),$$

where  $s_i$  and  $s_j$  are predicted positive-class scores, and  $n_+$  and  $n_-$  are the numbers of positive and negative samples. PR-AUC is the area under the precision-recall curve:

$$\text{PR-AUC} = \int_0^1 \text{Precision}(\text{Recall}) d\text{Recall}.$$

Binary F1 is computed for the positive class:

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

For California Housing, RMSE, MAE, and  $R^2$  are used. RMSE is defined as:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}.$$

MAE is defined as:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|.$$

The coefficient of determination,  $R^2$ , is defined as:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2},$$

where  $\bar{y}$  is the mean of the true target values.

#### F. XGBoost Baseline

XGBoost is used as the primary supervised baseline across all three datasets [1]. It is trained using only labeled data and does not use self-supervised pretraining. Hyperparameters are optimized separately for each dataset and each labeled-data fraction using Optuna with 50 trials [20].

The tuned hyperparameters include the number of estimators, maximum tree depth, learning rate, subsample ratio, column-sampling ratio, minimum child weight, L1 regularization, and L2 regularization. Table IV lists the Optuna search ranges used for the XGBoost baseline.

For each Optuna trial, the selected labeled training fraction is internally split into 80% training and 20% validation. The validation split is used for early stopping and hyperparameter selection. After hyperparameter selection, the final XGBoost

TABLE IV  
XGBOOST HYPERPARAMETER SEARCH RANGES

| Hyperparameter          | Search Range                                                     |
|-------------------------|------------------------------------------------------------------|
| <i>n_estimators</i>     | 200–1000, step 100                                               |
| <i>max_depth</i>        | 4–10 for Covertypes; 3–9 for Adult Income and California Housing |
| <i>learning_rate</i>    | 0.01–0.3, log scale                                              |
| <i>subsample</i>        | 0.6–1.0                                                          |
| <i>colsample_bytree</i> | 0.6–1.0                                                          |
| <i>min_child_weight</i> | 1–10                                                             |
| <i>reg_alpha</i>        | $10^{-8}$ –10, log scale                                         |
| <i>reg_lambda</i>       | $10^{-8}$ –10, log scale                                         |

model is trained again using the same internal early-stopping protocol and then evaluated on the fixed test split. Therefore, the test split is not used for Optuna tuning.

For Covertypes, XGBoost uses the multiclass objective `multi:softmax` with seven classes, and Optuna maximizes Macro-F1. For Adult Income, XGBoost uses `binary:logistic`, and Optuna maximizes ROC-AUC. Class imbalance is handled using `scale_pos_weight`, computed from the negative-to-positive class ratio. For California Housing, XGBoost uses `reg:squarederror`, and Optuna minimizes RMSE. GPU acceleration is enabled using histogram-based tree construction with CUDA where available.

#### G. TabNet on Covertypes

TabNet is evaluated on the Covertypes multiclass classification task [8]. TabNet uses sequential attention and sparse feature selection to learn which features are most relevant at each decision step.

The TabNet configuration follows the Covertypes-oriented settings used in the experiment, as shown in Table V.

TabNet pretraining uses masked feature reconstruction. Approximately 50% of input features are masked, and the model is trained to reconstruct the missing values without using labels. The pretrained encoder is then used to initialize supervised fine-tuning on the labeled fraction.

Several implementation adjustments are applied for stability. The pretraining learning rate is set lower than the fine-tuning learning rate, the pretraining batch size is capped at 2,048, and batch and virtual-batch sizes are adjusted according to the number of samples in each fraction. A fresh TabNetPretrainer is created for each fraction to avoid weight sharing across fractions.

In the final notebook, the fixed held-out Covertypes test split is passed to the `pytorch-tabnet eval_set` argument during pretraining and fine-tuning. This means the TabNet experiment uses the test split for early-stopping monitoring as well as final evaluation. This is a methodological limitation of the TabNet experiment. A stricter replication would create a validation split from the training fraction and reserve the test split only for final evaluation.

#### H. SAINT-Style Model on Adult Income

A simplified SAINT-style model is evaluated on Adult Income [7]. The original SAINT architecture combines feature-level self-attention and intersample attention. The implemen-

TABLE V  
TABNET CONFIGURATION

| Parameter                      | Value     |
|--------------------------------|-----------|
| $n_d$                          | 64        |
| $n_a$                          | 64        |
| $n\_steps$                     | 6         |
| $\gamma$                       | 1.5       |
| Mask type                      | sparsemax |
| $\lambda_{sparse}$             | 0.0001    |
| Fine-tuning learning rate      | 0.02      |
| Pretraining learning rate      | 0.005     |
| Fine-tuning epochs             | 300       |
| Pretraining epochs             | 200       |
| Fine-tuning patience           | 80        |
| Pretraining patience           | 50        |
| Maximum fine-tuning batch size | 16,384    |
| Maximum pretraining batch size | 2,048     |
| Ghost BatchNorm momentum       | 0.02      |

tation used in this study is built from scratch in PyTorch for compatibility and experimental control.

The model first projects each row into a single dense embedding of dimension 64. It then applies transformer-style blocks that include self-attention and row-level intersample attention. Because the model uses a single dense row embedding rather than separate feature tokens, it should be treated as a simplified SAINT-style implementation rather than a full reproduction of the original SAINT architecture.

The main hyperparameters of the simplified SAINT-style model are listed in Table VI.

SAINT-style self-supervised pretraining is performed on the full Adult Income training split without labels. Two objectives are used. The denoising objective randomly zeros 30% of input features and trains the model to reconstruct the original feature vector using mean squared error. The contrastive objective creates two augmented views of each row by adding Gaussian noise to the input features. The model then uses NT-Xent loss to pull together embeddings from the same row and push apart embeddings from different rows.

The total pretraining loss is:

$$L_{pretrain} = L_{denoise} + L_{contrastive}.$$

After pretraining, the full model is fine-tuned on the selected labeled fraction using weighted cross-entropy. All model parameters are updated during fine-tuning. The class weights are computed from the selected labeled fraction:

$$w = \left[ 1, \frac{N_{negative}}{N_{positive}} \right],$$

TABLE VI  
SAINT-STYLE MODEL HYPERPARAMETERS

| Parameter                | Value     |
|--------------------------|-----------|
| Embedding dimension      | 64        |
| Attention heads          | 8         |
| Layers                   | 3         |
| Dropout                  | 0.1       |
| Batch size               | 256       |
| Learning rate            | $10^{-4}$ |
| Pretraining epochs       | 100       |
| Fine-tuning epochs       | 100       |
| Pretraining weight decay | $10^{-4}$ |
| Fine-tuning weight decay | $10^{-5}$ |

where  $N_{negative}$  and  $N_{positive}$  are the numbers of negative and positive samples in the selected fraction. This gives higher weight to the minority positive class.

The SAINT-style experiments were run in Google Colab on CPU due to free-tier GPU availability. Results were saved after each fraction to allow recovery from notebook disconnections.

#### I. TabPFN v2 on California Housing

TabPFN v2 is evaluated on California Housing as a pre-trained tabular foundation model [12], [13]. Unlike TabNet, SAINT, and T-JEPA, TabPFN is not trained through target-dataset SSL. It is pretrained offline on synthetic tabular tasks and performs prediction through in-context learning.

The estimator used in the experiment is TabPFNRegressor. Although the notebook and CSV label this experiment as `realtabpfv2`, the evaluated model is the base TabPFN v2 regressor rather than Real-TabPFN. Therefore, the paper refers to this model as TabPFN v2.

For each labeled fraction, TabPFNRegressor receives the selected California Housing training samples and targets as context through `fit()`. This step does not perform conventional gradient-based training. Predictions are then produced for the fixed test split through the model’s in-context inference mechanism.

The TabPFN v2 configuration used in this experiment is shown in Table VII.

The `ignore_pretraining_limits=True` option is used because the 100% California Housing training split exceeds the model’s recommended sample limit. No external StandardScaler is applied to the raw TabPFN v2 experiment because TabPFN performs internal preprocessing.

TABLE VII  
TABPFN v2 CONFIGURATION

| Parameter                              | Value           |
|----------------------------------------|-----------------|
| Estimator                              | TabPFNRegressor |
| Device                                 | CUDA            |
| Random state                           | 42              |
| <code>ignore_pretraining_limits</code> | True            |
| <code>n_estimators</code>              | 8               |

#### J. T-JEPA $\rightarrow$ TabPFN Pipeline

The T-JEPA  $\rightarrow$  TabPFN pipeline is a two-stage SSL-to-foundation-model experiment on California Housing. First, T-JEPA is pretrained on the unlabeled training features. Then, the learned embeddings are passed to TabPFNRegressor instead of the raw input features.

T-JEPA is an augmentation-free SSL method that predicts masked feature representations in latent space rather than reconstructing raw feature values [14]. The implementation contains a feature embedder, a context encoder, a target encoder, and a predictor. Numerical features are standardized before T-JEPA pretraining.

The main T-JEPA configuration used before embedding extraction is reported in Table VIII.

At each training step, four context-target mask pairs are sampled. The context subset contains a randomly selected 40%–75% of the features. The target subset contains one to three features selected from the remaining features. The context encoder processes only the context features, while the target encoder processes all features and selects the target positions. The target encoder is updated by exponential moving average rather than direct gradient descent.

The EMA update is:

$$\theta_{\text{target}} \leftarrow m\theta_{\text{target}} + (1 - m)\theta_{\text{context}},$$

where  $m$  increases linearly from 0.996 to 1.0 over training epochs. A learnable regularization token is appended to all encoder inputs to prevent representation collapse during EMA-coupled training. Without this token, the EMA coupling between context and target encoders can cause embeddings to converge to trivial constant representations.

The predictor receives the context representation and learnable mask tokens, then predicts the target latent representation. The T-JEPA loss is the mean squared error between predicted and target latent representations:

$$L_{T-JEPA} = \frac{1}{K} \sum_{k=1}^K \|g_{\phi}(f_{\theta}(x_{C_k}), T_k) - f_{\bar{\theta}}(x)_{T_k}\|_2^2,$$

where  $K$  is the number of prediction masks,  $C_k$  is the context feature subset,  $T_k$  is the target feature subset,  $f_{\theta}$  is the context encoder,  $f_{\bar{\theta}}$  is the EMA target encoder, and  $g_{\phi}$  is the predictor.

T-JEPA is pretrained once on the full unlabeled California Housing training split. It is not retrained separately for each labeled-data fraction. Pretraining converged successfully, with

TABLE VIII  
T-JEPA CONFIGURATION

| Parameter                  | Value     |
|----------------------------|-----------|
| Number of features         | 8         |
| Encoder hidden dimension   | 64        |
| Encoder heads              | 8         |
| Encoder layers             | 3         |
| Feed-forward dimension     | 256       |
| Predictor hidden dimension | 32        |
| Predictor heads            | 4         |
| Predictor layers           | 4         |
| Dropout                    | 0.0       |
| Batch size                 | 512       |
| Pretraining epochs         | 200       |
| Early-stopping patience    | 50        |
| Warmup epochs              | 10        |
| Learning rate              | $10^{-4}$ |
| Weight decay               | $10^{-4}$ |
| EMA start                  | 0.996     |
| EMA end                    | 1.0       |
| Prediction masks per batch | 4         |

the loss decreasing from 0.4917 at epoch 20 to a best loss of 0.2908 before early stopping at epoch 153 of 200. This confirms that the encoder completed non-collapsed representation pretraining before the downstream TabPFN experiments.

After pretraining, embeddings are extracted from the context encoder. The encoder output has shape:

$$(N, 8, 64),$$

where  $N$  is the number of samples, 8 is the number of California Housing features, and 64 is the hidden dimension. Since TabPFN requires a two-dimensional input matrix, the feature-level embeddings are mean-pooled across the feature dimension:

$$X_{\text{emb}} = \frac{1}{8} \sum_{j=1}^8 h_j,$$

resulting in a 64-dimensional embedding for each sample:

$$(N, 8, 64) \rightarrow (N, 64).$$

A StandardScaler is fitted on the training embeddings and applied to the test embeddings. For each labeled-data frac-

TABLE IX  
COMPUTATIONAL ENVIRONMENT SUMMARY

| Experiment                  | Environment     | Hardware                   | Main Libraries                |
|-----------------------------|-----------------|----------------------------|-------------------------------|
| XGBoost                     | Local / VS Code | NVIDIA RTX 3050, CUDA 12.5 | xgboost, optuna, scikit-learn |
| TabNet                      | Kaggle          | P100 GPU                   | pytorch-tabnet, scikit-learn  |
| SAINT-style model           | Google Colab    | CPU                        | PyTorch, scikit-learn         |
| TabPFN v2                   | Local / VS Code | NVIDIA RTX 3050, CUDA 12.5 | tabpfn, scikit-learn          |
| T-JEPA $\rightarrow$ TabPFN | Local / VS Code | NVIDIA RTX 3050, CUDA 12.5 | PyTorch, tabpfn, scikit-learn |

tion, TabPFNRegressor is then fitted on the selected training embeddings and evaluated on the fixed test embeddings.

### K. Computational Environment

The experiments were run across local, Kaggle, and Google Colab environments. Table IX summarizes the hardware and main libraries used for each experiment.

The notebooks save results to CSV files after each fraction to reduce the risk of losing completed experiments during long runs or notebook interruptions.

## V. RESULTS AND DISCUSSION

### A. Overall Results Summary

The results show that there is no universal labeled-data crossover point where SSL-based deep tabular models consistently outperform XGBoost across all datasets. Instead, the outcome depends strongly on the dataset, task type, model architecture, and pretraining strategy.

XGBoost remains highly competitive on the two classification datasets, especially Covertypes and Adult Income. TabNet does not outperform XGBoost on Covertypes under the primary accuracy metric, and its SSL pretraining is unstable across most labeled-data fractions. On Adult Income, the SAINT-style model only surpasses XGBoost under the primary ROC-AUC metric in the most extreme low-label setting, where only 1% of the training labels are available. On California Housing, TabPFN v2 provides the strongest overall results and outperforms XGBoost at every labeled-data fraction. However, adding T-JEPA embeddings before TabPFN does not improve over raw TabPFN v2, suggesting that external SSL representations are not automatically beneficial for a pretrained foundation model.

These results support the main conclusion of this study: SSL can help in extreme low-label settings, but it is not a guaranteed replacement for a tuned tree-based model. In contrast, pretrained tabular foundation models may provide a more reliable improvement direction, at least for small-to-medium tabular regression tasks such as California Housing. Figure 1 illustrates the XGBoost baseline trends across the three datasets and labeled-data fractions.

### B. Covertypes: TabNet vs XGBoost

Table X compares TabNet and XGBoost on the Covertypes multiclass classification task using accuracy and Macro-F1.

Figure 2 visualizes the same Covertypes comparison as performance curves, highlighting how both accuracy and Macro-F1 change as the labeled-data fraction decreases.

XGBoost achieves higher accuracy than TabNet at every labeled-data fraction. This means that under the primary

**XGBoost Baseline — Performance vs Data Fraction**

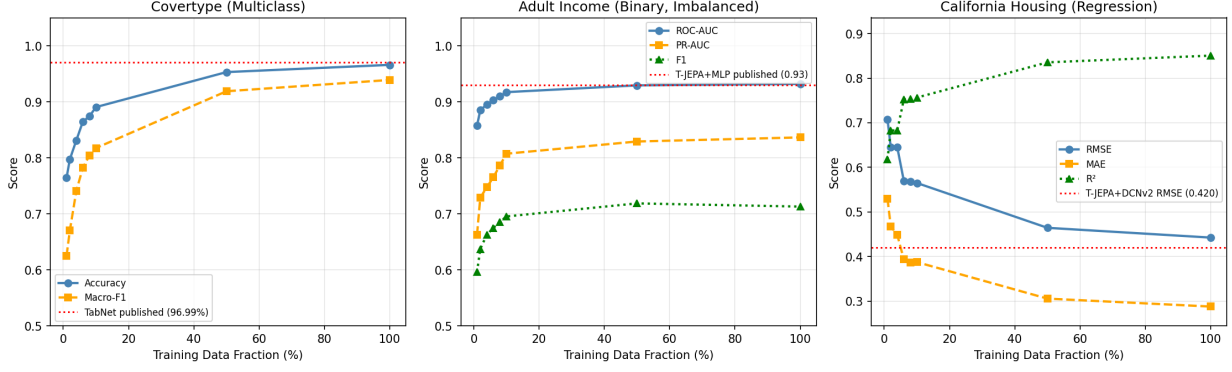


Fig. 1. XGBoost baseline performance across labeled-data fractions for Covertype, Adult Income, and California Housing.

**TabNet vs XGBoost — Covertype**

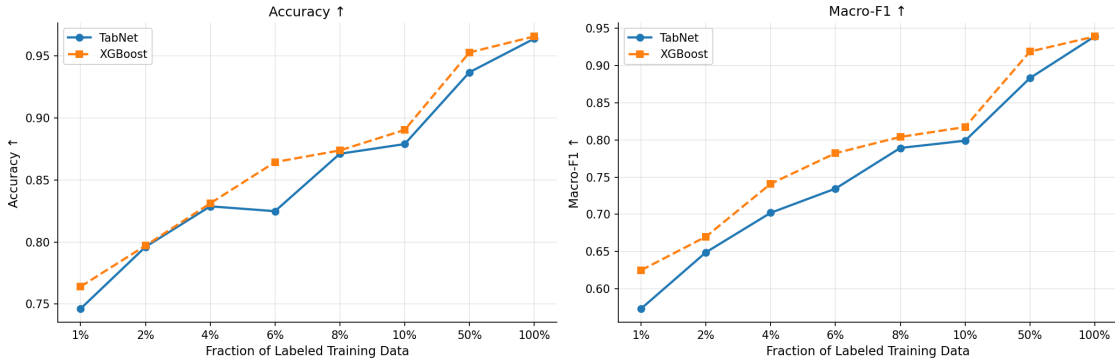


Fig. 2. TabNet and XGBoost performance on Covertype across labeled-data fractions.

TABLE X  
COVERTYPE RESULTS: TABNET VS XGBOOST

| Labeled Fraction | TabNet Accuracy | XGBoost Accuracy | TabNet Macro-F1 | XGBoost Macro-F1 |
|------------------|-----------------|------------------|-----------------|------------------|
| 100%             | 0.9640          | <b>0.9658</b>    | <b>0.9390</b>   | 0.9389           |
| 50%              | 0.9369          | <b>0.9529</b>    | 0.8830          | <b>0.9187</b>    |
| 10%              | 0.8790          | <b>0.8904</b>    | 0.7989          | <b>0.8173</b>    |
| 8%               | 0.8712          | <b>0.8739</b>    | 0.7892          | <b>0.8040</b>    |
| 6%               | 0.8249          | <b>0.8645</b>    | 0.7346          | <b>0.7822</b>    |
| 4%               | 0.8288          | <b>0.8312</b>    | 0.7021          | <b>0.7411</b>    |
| 2%               | 0.7961          | <b>0.7971</b>    | 0.6490          | <b>0.6700</b>    |
| 1%               | 0.7463          | <b>0.7641</b>    | 0.5733          | <b>0.6249</b>    |

Covertype metric, TabNet does not reach a crossover point against XGBoost. The Macro-F1 results show a similar trend at reduced-data fractions. The only exception is the full-data setting, where TabNet achieves a Macro-F1 of 0.9390 compared with 0.9389 for XGBoost. This difference is extremely small and should be interpreted as a practical tie rather than a meaningful advantage.

A likely reason for XGBoost’s stronger Covertype performance is that the dataset contains many structured numerical and binary indicator features. Gradient-boosted trees are especially effective at learning threshold-based rules and nonlinear interactions in tabular feature spaces. As the training fraction increases, XGBoost benefits directly from more labeled examples and can fit more reliable tree splits.

TabNet, on the other hand, depends on neural representation

learning and attention-based feature selection. Although this design is powerful in principle, it does not guarantee better performance on every tabular dataset. In this experiment, TabNet’s masked feature reconstruction pretraining was marked successful only at the 1% fraction, where `best_pretrain_epoch` reached 10. At all other fractions, pretraining either found no improvement at all, with `best_pretrain_epoch` = 0 from 100% through 6%, or found only marginal improvement before early stopping, with epochs 1 and 3 at 4% and 2%, respectively. Therefore, the final TabNet model in all cases except 1% mainly reflects supervised TabNet training rather than successful SSL-enhanced learning.

This instability is important because Covertype is large and heterogeneous, with mixed continuous and binary indicator features. Masked reconstruction can become difficult when feature scales and feature meanings differ strongly. Reconstructing masked inputs also does not directly optimize class separation. As a result, the SSL objective may fail to produce representations that improve classification performance. These results suggest that TabNet’s masked reconstruction SSL is not automatically stable at Covertype scale, even though TabNet itself remains competitive at high labeled-data fractions.

TABLE XI  
ADULT INCOME RESULTS: SAINT-STYLE MODEL VS XGBOOST

| Labeled Fraction | SAINT ROC-AUC | XGBoost ROC-AUC | SAINT PR-AUC  | XGBoost PR-AUC | SAINT F1      | XGBoost F1    |
|------------------|---------------|-----------------|---------------|----------------|---------------|---------------|
| 100%             | 0.9091        | <b>0.9317</b>   | 0.7795        | <b>0.8363</b>  | 0.6783        | <b>0.7128</b> |
| 50%              | 0.9073        | <b>0.9292</b>   | 0.7753        | <b>0.8290</b>  | 0.6749        | <b>0.7185</b> |
| 10%              | 0.8948        | <b>0.9171</b>   | 0.7422        | <b>0.8072</b>  | 0.6577        | <b>0.6952</b> |
| 8%               | 0.8910        | <b>0.9095</b>   | 0.7359        | <b>0.7862</b>  | 0.6539        | <b>0.6854</b> |
| 6%               | 0.8919        | <b>0.9025</b>   | 0.7383        | <b>0.7652</b>  | 0.6580        | <b>0.6744</b> |
| 4%               | 0.8893        | <b>0.8950</b>   | 0.7308        | <b>0.7472</b>  | 0.6529        | <b>0.6629</b> |
| 2%               | 0.8731        | <b>0.8851</b>   | 0.6827        | <b>0.7289</b>  | <b>0.6392</b> | 0.6369        |
| 1%               | <b>0.8596</b> | 0.8571          | <b>0.6691</b> | 0.6626         | <b>0.6110</b> | 0.5960        |

### C. Adult Income: SAINT-Style Model vs XGBoost

Table XI compares the SAINT-style model and XGBoost on Adult Income using ROC-AUC, PR-AUC, and F1.

Figure 3 shows the Adult Income performance curves, including the 1% labeled-data setting where the SAINT-style model slightly exceeds XGBoost.

On Adult Income, XGBoost outperforms the SAINT-style model on ROC-AUC from 100% down to 2% labeled data. However, at the 1% labeled-data fraction, the SAINT-style model slightly surpasses XGBoost on all three reported metrics. Under the primary metric, ROC-AUC, SAINT achieves 0.8596 compared with 0.8571 for XGBoost. The improvement at 1% is small but directionally consistent across all three metrics. At the 2% fraction, SAINT also gives a slightly higher F1 score than XGBoost, but XGBoost remains stronger on ROC-AUC and PR-AUC, so this is not treated as the main crossover point.

The 1% result suggests that the SAINT-style model benefits most when labeled data is extremely scarce. This is plausible because the model uses self-supervised pretraining on the full unlabeled training features before fine-tuning on the limited labeled subset. The denoising objective encourages the model to learn relationships between features, while the contrastive objective encourages similar representations for augmented views of the same sample. Weighted cross-entropy also helps address class imbalance during fine-tuning.

However, XGBoost remains stronger across most fractions. Adult Income contains many categorical and threshold-like patterns, which are well suited to gradient-boosted trees. Once enough labeled samples are available, XGBoost can learn strong supervised decision rules directly. In contrast, the SAINT-style model must learn a neural representation from encoded and scaled tabular inputs. The implementation used here is also a simplified SAINT-style model rather than a full reproduction of the original SAINT architecture. It uses Gaussian-noise views for contrastive learning rather than the original SAINT paper’s stronger CutMix and mixup augmentations. These implementation choices may limit the benefit of SSL at larger labeled-data fractions.

Overall, the Adult Income results support the idea that SSL is most useful when labels are extremely scarce. The SAINT-style model does not generally replace XGBoost, but it becomes competitive and slightly stronger at the 1% labeled-data level.

### D. California Housing: XGBoost vs TabPFN v2 vs T-JEPA→TabPFN

Table XII compares XGBoost, TabPFN v2, and T-JEPA→TabPFN on California Housing using RMSE, MAE, and  $R^2$ . Lower RMSE and MAE indicate better performance, while higher  $R^2$  indicates better performance.

Figure 4 plots the California Housing comparison between TabPFN v2 and XGBoost, showing TabPFN v2’s lower error and higher  $R^2$  across all fractions.

Figure 5 compares raw TabPFN v2, T-JEPA→TabPFN, and XGBoost, making the inconsistent benefit of T-JEPA embeddings visible across fractions.

TabPFN v2 is the strongest model on California Housing. It achieves lower RMSE and MAE and higher  $R^2$  than XGBoost at every labeled-data fraction. TabPFN v2 outperforms XGBoost at every tested fraction, from 1% through 100%, with no fraction at which XGBoost achieves lower RMSE.

A likely reason for TabPFN v2’s strong performance is that it is not trained from scratch on each fraction. Instead, it is a pretrained tabular foundation model that uses in-context learning. The labeled training samples are provided as context, and prediction is performed using a pretrained transformer. This gives TabPFN v2 a strong prior over tabular prediction tasks before seeing the California Housing data. California Housing is also relatively small and low-dimensional, with only 8 numerical features, which is a favorable setting for TabPFN-style inference.

XGBoost remains competitive, especially at larger fractions, but it is purely supervised. It must learn each regression function from the labeled samples available in the current fraction. When labels are limited, this restricts how much structure it can learn. TabPFN v2, by contrast, benefits from prior meta-training on many synthetic tabular tasks, which likely explains its better data efficiency.

The T-JEPA→TabPFN pipeline gives a more nuanced result. T-JEPA pretraining converged successfully before downstream evaluation, so the negative outcome is not simply due to failed pretraining. However, T-JEPA→TabPFN does not improve over raw TabPFN v2 at any fraction. TabPFN v2 consistently achieves lower RMSE and MAE and higher  $R^2$  than the T-JEPA-enhanced pipeline.

Compared with XGBoost, T-JEPA→TabPFN is mixed. Under RMSE, it outperforms XGBoost at 10%, 8%, 4%, and 2%, but it does not outperform XGBoost at 100%, 50%, 6%, or 1%. Therefore, although T-JEPA embeddings can sometimes improve over XGBoost, the improvement is inconsistent and does not form a stable crossover pattern.

A plausible explanation is that TabPFN v2 is already optimized to operate on raw tabular inputs. Feeding it external SSL embeddings changes the input distribution from raw features into learned latent representations. This may remove useful feature identity information, especially because the final T-JEPA representation is obtained by mean-pooling the encoder output from  $(N, 8, 64)$  to  $(N, 64)$ . Mean-pooling creates a compact sample-level embedding, but it may also discard feature-specific structure that TabPFN could have used directly

SAINT vs XGBoost — Adult Income (Binary, Imbalanced)

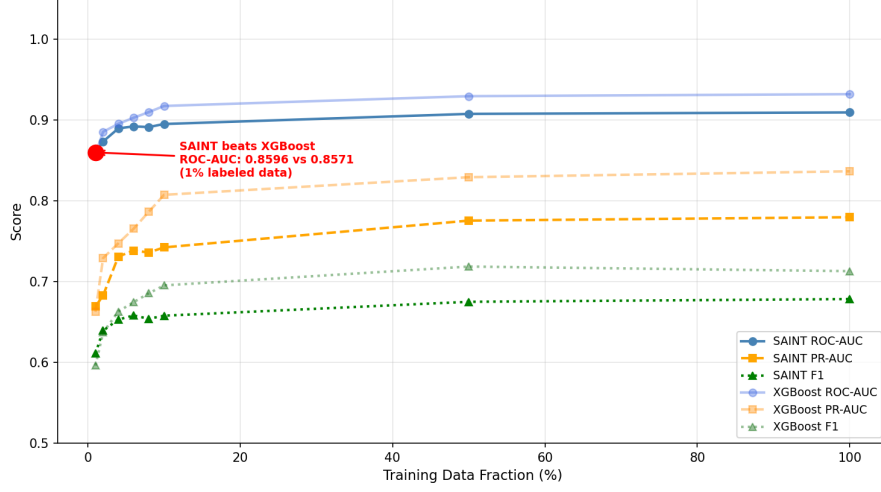


Fig. 3. SAINT-style model and XGBoost performance on Adult Income across labeled-data fractions.

TABLE XII  
CALIFORNIA HOUSING RESULTS: XGBOOST VS TABPFN v2 VS T-JEPA→TABPFN

| Labeled Fraction | XGBoost RMSE | TabPFN v2 RMSE | T-JEPA→TabPFN RMSE | XGBoost MAE | TabPFN v2 MAE | T-JEPA→TabPFN MAE | XGBoost $R^2$ | TabPFN v2 $R^2$ | T-JEPA→TabPFN $R^2$ |
|------------------|--------------|----------------|--------------------|-------------|---------------|-------------------|---------------|-----------------|---------------------|
| 100%             | 0.4424       | <b>0.3905</b>  | 0.4499             | 0.2877      | <b>0.2357</b> | 0.2836            | 0.8506        | <b>0.8836</b>   | 0.8455              |
| 50%              | 0.4643       | <b>0.4088</b>  | 0.4733             | 0.3055      | <b>0.2475</b> | 0.2990            | 0.8355        | <b>0.8725</b>   | 0.8290              |
| 10%              | 0.5649       | <b>0.4632</b>  | 0.5425             | 0.3874      | <b>0.2935</b> | 0.3554            | 0.7565        | <b>0.8363</b>   | 0.7754              |
| 8%               | 0.5686       | <b>0.4829</b>  | 0.5613             | 0.3867      | <b>0.3087</b> | 0.3698            | 0.7533        | <b>0.8220</b>   | 0.7596              |
| 6%               | 0.5698       | <b>0.4922</b>  | 0.5710             | 0.3936      | <b>0.3178</b> | 0.3781            | 0.7523        | <b>0.8151</b>   | 0.7512              |
| 4%               | 0.6449       | <b>0.5153</b>  | 0.5891             | 0.4484      | <b>0.3342</b> | 0.3900            | 0.6826        | <b>0.7974</b>   | 0.7352              |
| 2%               | 0.6448       | <b>0.5634</b>  | 0.6289             | 0.4675      | <b>0.3774</b> | 0.4245            | 0.6828        | <b>0.7578</b>   | 0.6982              |
| 1%               | 0.7078       | <b>0.6091</b>  | 0.7088             | 0.5299      | <b>0.4146</b> | 0.4613            | 0.6177        | <b>0.7168</b>   | 0.6166              |

Real-TabPFN vs XGBoost — California Housing

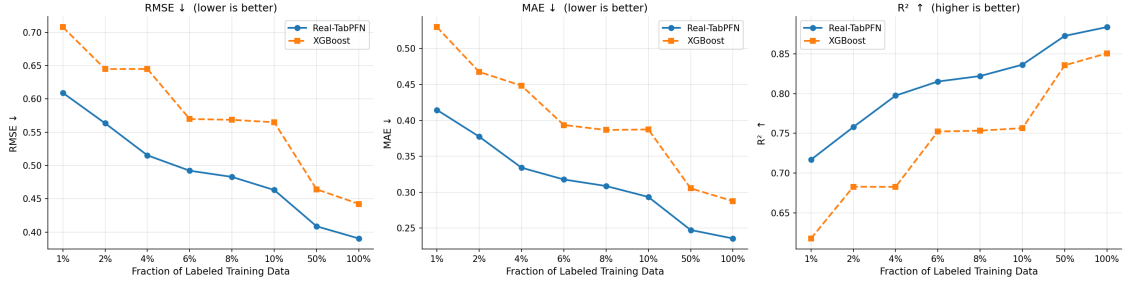


Fig. 4. TabPFN v2 and XGBoost performance on California Housing across labeled-data fractions.

T-JEPA → TabPFN vs TabPFN (raw) vs XGBoost — California Housing

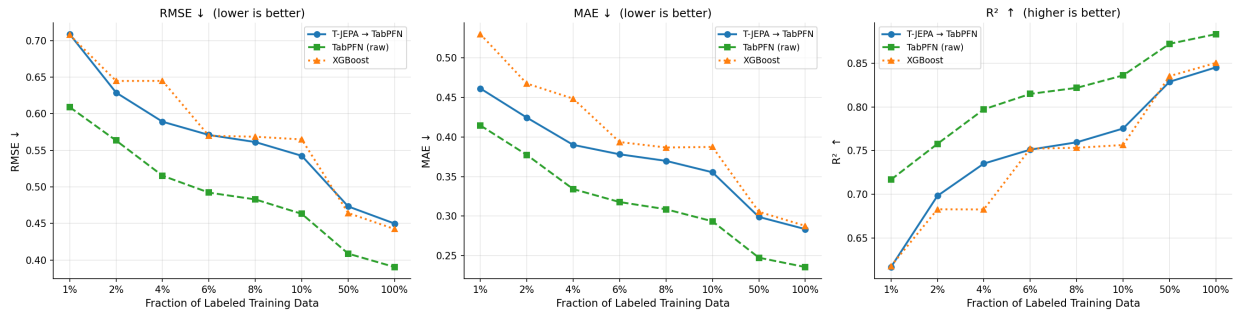


Fig. 5. T-JEPA→TabPFN, raw TabPFN v2, and XGBoost performance on California Housing across labeled-data fractions.

from the original input columns. In other words, T-JEPA may learn useful representations, but those representations are not necessarily better suited for TabPFN than the raw California Housing features.

This suggests that combining target-dataset SSL with foundation-model inference is not automatically beneficial. The two approaches may capture overlapping information, or the SSL embedding may shift the data away from the distribution expected by the foundation model.

#### E. Crossover Point Analysis

The crossover point is defined as the smallest labeled-data fraction at which the deep model performs at least as well as XGBoost under the primary metric for that dataset. Table XIII summarizes the resulting crossover point for each evaluated deep model or pipeline.

The crossover analysis shows that SSL does not provide a universal advantage. TabNet does not cross over XGBoost on Covertypes under accuracy. SAINT crosses over only at 1% labeled data on Adult Income, supporting the idea that SSL is most useful when labels are extremely scarce. TabPFN v2 is the clearest success case: it remains stronger than XGBoost from 1% through 100% labeled data, with no tested fraction where XGBoost achieves lower RMSE. T-JEPA→TabPFN shows that simply adding an SSL representation before a foundation model does not guarantee improvement, since it beats XGBoost only at some fractions and never beats raw TabPFN v2.

#### F. Summary of Findings

The main findings are as follows. First, XGBoost remains a very strong baseline for tabular data and should not be treated as an easy model to outperform. On Covertypes, it outperforms TabNet in accuracy at every labeled-data fraction. On Adult Income, it also outperforms SAINT across most fractions under ROC-AUC and PR-AUC.

Second, SSL provides limited but visible benefit in the most label-scarce setting. The SAINT-style model slightly outperforms XGBoost on Adult Income at 1% labeled data, and this improvement is directionally consistent across ROC-AUC, PR-AUC, and F1.

Third, TabNet’s masked reconstruction SSL is not stable on Covertypes in this implementation. This negative result is important because it shows that SSL success on one tabular dataset does not automatically transfer to another, especially when the dataset is large and heterogeneous.

Fourth, TabPFN v2 is the strongest model in the California Housing experiment. Its consistent improvement over XGBoost across all eight labeled-data fractions suggests that pretrained tabular foundation models may be a more reliable direction than dataset-specific SSL for some tabular regression tasks.

Finally, the proposed T-JEPA→TabPFN pipeline does not improve over raw TabPFN v2. Although T-JEPA pretraining converged, the learned embeddings did not provide additional benefit to TabPFN. This suggests that TabPFN’s in-context

learning may already capture much of the useful structure available in the raw features, or that external SSL embeddings are not aligned with the type of input representation TabPFN expects.

## VI. CONCLUSION, LIMITATIONS, AND FUTURE WORK

### A. Conclusion

This study investigated the labeled-data crossover point at which self-supervised and pretrained deep tabular models begin to outperform a properly tuned XGBoost baseline across three benchmark datasets and eight labeled-data fractions. The central finding is that no universal crossover point exists. The benefit of SSL depends heavily on the dataset, task type, model architecture, and stability of the pretraining objective.

For practitioners choosing between XGBoost and deep tabular learning, these results suggest three practical guidelines. First, a well-tuned XGBoost baseline should always be established before investing in SSL pretraining, as it remains competitive or superior in most labeled-data regimes studied here. Second, SSL pretraining appears most useful in the most extreme low-label setting tested in this study, especially when the model architecture supports stable pretraining on the target dataset. This is shown most clearly by the SAINT-style model on Adult Income, which only surpasses XGBoost at the 1% labeled-data fraction. Third, tabular foundation models such as TabPFN v2 represent a qualitatively different direction from target-dataset SSL. Their advantage does not come from learning a new representation through SSL on the target dataset, but from using a pretrained prior over tabular task distributions. In the California Housing experiment, this prior allows TabPFN v2 to outperform XGBoost at all eight tested labeled-data fractions.

The proposed T-JEPA→TabPFN pipeline did not improve over raw TabPFN v2. However, this negative result is informative. Since T-JEPA pretraining converged before downstream evaluation, the result suggests that foundation-model inference and external SSL representation learning may address overlapping aspects of feature structure. It also shows that a naive composition of SSL embeddings with TabPFN-style in-context learning is not automatically beneficial.

Overall, the findings suggest that SSL should not be treated as a guaranteed replacement for tuned tree-based models on tabular data. Instead, its usefulness depends on the interaction between dataset structure, label availability, model design, and pretraining stability. Among the methods tested, TabPFN v2 provides the strongest evidence that pretrained tabular foundation models may offer a more reliable path for improving performance in some tabular regression settings.

### B. Limitations

Several limitations should be considered when interpreting the results.

**Dataset scope.** Three datasets were evaluated: one multiclass classification task, one imbalanced binary classification task, and one regression task. The findings may not generalize to

TABLE XIII  
CROSSOVER POINT SUMMARY

| Dataset            | Deep Model / Pipeline | Primary Metric | Crossover Point vs XGBoost                         | Interpretation                                                                        |
|--------------------|-----------------------|----------------|----------------------------------------------------|---------------------------------------------------------------------------------------|
| Covertypes         | TabNet                | Accuracy       | No crossover                                       | XGBoost has higher accuracy at every fraction                                         |
| Adult Income       | SAINT-style model     | ROC-AUC        | 1%                                                 | SAINT only surpasses XGBoost under ROC-AUC in the most extreme low-label setting      |
| California Housing | TabPFN v2             | RMSE           | All tested fractions (1%–100%)                     | TabPFN v2 outperforms XGBoost at all eight tested fractions — the only model to do so |
| California Housing | T-JEPA→TabPFN         | RMSE           | 2% by strict fraction definition, but inconsistent | T-JEPA→TabPFN beats XGBoost at some low-to-mid fractions but not consistently         |

datasets with very high dimensionality, strong temporal structure, heavy missing-value rates, different categorical feature structures, or datasets much larger than Covertypes.

**Model scope.** Each deep learning model was paired with a specific dataset based on benchmark relevance and computational feasibility. This means that full cross-model comparisons on every dataset were not performed. For example, SAINT and TabNet were not both evaluated on Covertypes, and TabPFN v2 was not evaluated on the classification datasets. These comparisons remain open questions.

**SAINT implementation.** The SAINT-style model used in this study is a simplified from-scratch implementation rather than a full reproduction of the original SAINT architecture. It uses Gaussian-noise contrastive views rather than the original CutMix and mixup augmentations, and it projects each row into a single dense embedding rather than fully preserving separate feature-token interactions. It was also trained on CPU due to Google Colab free-tier constraints, which limited the ability to perform broader tuning or longer repeated experiments. These differences may limit the upper bound of SAINT performance in this study.

**TabNet SSL stability.** TabNet’s masked reconstruction pretraining was marked successful only at the 1% fraction. At all other fractions, pretraining either found no improvement or only marginal improvement before early stopping. Therefore, the TabNet results mostly reflect supervised TabNet training rather than successful SSL-enhanced TabNet. This limits the conclusions that can be drawn about TabNet SSL benefit specifically on Covertypes.

**TabNet validation protocol.** In the final TabNet notebook, the fixed Covertypes test split was passed to the `eval_set` argument during pretraining and fine-tuning. This means the test split was used for early-stopping monitoring as well as final evaluation. A stricter replication should create a validation split from the training fraction and reserve the test split only for final evaluation.

**Adult Income preprocessing.** In the implemented Adult Income notebooks, categorical features were label-encoded before the train-test split. StandardScaler was fitted after the split using only the training data. A stricter future replication should fit categorical encoders using only the training split and then apply the fitted encoders to the test split.

**T-JEPA projection.** The T-JEPA→TabPFN pipeline uses mean pooling to project the encoder output from  $(N, 8, 64)$

to  $(N, 64)$ . This is a practical design choice that avoids using a supervised projection layer, but it may discard feature-specific structure. Alternative projection strategies, such as flattening, attention-based pooling, or a dedicated unsupervised projection head, may produce different results and were not evaluated.

**Single random seed.** All experiments use a fixed random seed of 42 for reproducibility. Results may vary across seeds, especially at very small labeled-data fractions. This study does not report mean and standard deviation across multiple random runs, so small performance differences should be interpreted cautiously.

**Computational constraints.** The experiments were run across local, Kaggle, and Google Colab environments with different hardware availability. These constraints affected the feasible training time, model tuning depth, and number of repeated runs, particularly for the SAINT-style model and TabNet pretraining.

### C. Future Work

The findings of this study suggest several directions for future investigation.

**Extending the SSL-to-foundation-model pipeline.** The T-JEPA→TabPFN pipeline introduced here is a first attempt at combining SSL representation learning with foundation-model inference. Future work could explore alternative SSL methods as preprocessors for TabPFN, such as SubTab, SCARF, and VIME, to determine whether the negative result is specific to T-JEPA or generalizes across SSL objectives [4]–[6]. Future work could also investigate learnable projection heads trained with unsupervised objectives to better align the SSL embedding space with TabPFN’s expected input distribution.

**Broader dataset coverage.** This study used three datasets covering specific task types. Future work should evaluate crossover behavior on a wider range of datasets, including high-dimensional datasets, datasets with stronger class imbalance, datasets with substantial missing values, and datasets with more categorical features. This would test whether the conclusions hold under more challenging tabular conditions.

**Full SAINT reproduction.** The simplified SAINT-style model used here differs from the original architecture in several ways. A full reproduction with GPU training, CutMix and mixup augmentations, separate feature tokenization, and more extensive hyperparameter tuning could yield stronger SSL per-

formance on Adult Income and potentially shift the crossover point to larger labeled-data fractions.

**Improved TabNet SSL evaluation.** Future work should evaluate TabNet SSL with a stricter validation protocol and additional stability controls. This includes using a validation split from the training fraction rather than the fixed test split for early stopping, testing multiple random seeds, and examining whether TabNet SSL becomes more stable when Covertypes is subsampled to controlled dataset sizes before pretraining. This would help separate the effect of dataset scale from the effect of supervised fine-tuning fraction.

**Alternative T-JEPA embedding projections.** The current T-JEPA→TabPFN pipeline uses mean pooling to convert feature-level embeddings into sample-level embeddings. Future work could compare mean pooling with flattening, attention pooling, unsupervised bottleneck projection, or feature-preserving projection strategies. This would help determine whether T-JEPA failed to improve TabPFN because of the SSL representation itself or because of the projection method used before TabPFN inference.

**Variance analysis.** Running each experiment across multiple random seeds and reporting mean and standard deviation would strengthen the reliability of the crossover point estimates. This is especially important at low labeled-data fractions where the performance gap between models can be small, such as the 1% Adult Income comparison between SAINT and XGBoost.

**Foundation models on classification tasks.** TabPFN v2 was evaluated only on California Housing regression in this study. Future work could evaluate TabPFN v2 on the classification datasets as well, allowing direct comparison against TabNet, SAINT, and XGBoost under the same fraction-scaling protocol.

## REFERENCES

- [1] T. Chen and C. Guestrin, “Xgboost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2016.
- [2] Y. Gorishniy, I. Rubachev, V. Khrulkov, and A. Babenko, “Revisiting deep learning models for tabular data,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [3] R. Shwartz-Ziv and A. Armon, “Tabular data: Deep learning is not all you need,” *Information Fusion*, 2022.
- [4] J. Yoon, J. Jordon, and M. van der Schaar, “Vime: Extending the success of self- and semi-supervised learning to tabular domain,” arXiv preprint arXiv:1907.05556, 2019.
- [5] D. Bahri, Y. Jiang, Y. Tay, and D. Metzler, “Scarf: Self-supervised contrastive learning using random feature corruption,” arXiv preprint arXiv:2106.15147, 2021.
- [6] M. Ucar, E. Hajiramezanali, L. Edwards, Y. Liang, K.-T. Chang, N. Sinnott-Armstrong, J. Dy, L. Getoor, S. Saria, and J. Wiens, “Subtab: Subsetting features of tabular data for self-supervised representation learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [7] G. Somepalli, M. Goldblum, A. Schwarzschild, C. Bruss, and T. Goldstein, “Saint: Improved neural networks for tabular data via row attention and contrastive pre-training,” arXiv preprint arXiv:2106.01342, 2021.
- [8] S. Ö. Arik and T. Pfister, “Tabnet: Attentive interpretable tabular learning,” in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2021.
- [9] J. A. Blackard and D. J. Dean, “Covertypes,” UCI Machine Learning Repository, 1998.
- [10] B. Becker and R. Kohavi, “Adult,” UCI Machine Learning Repository, 1996.
- [11] R. K. Pace and R. Barry, “Sparse spatial autoregressions,” *Statistics & Probability Letters*, vol. 33, no. 3, pp. 291–297, 1997.
- [12] N. Hollmann, S. Müller, L. Purucker, A. Krishnakumar, U. Köthe, and F. Hutter, “TabPFN: A transformer that solves small tabular classification problems in a second,” arXiv preprint arXiv:2207.01848, 2022.
- [13] A. Garg, M. Ali, N. Hollmann, L. Purucker, S. Müller, and F. Hutter, “Real-tabPFN: Improving tabular foundation models via continued pre-training with real-world data,” arXiv preprint arXiv:2507.03971, 2025.
- [14] H. Thimonier, J. L. De Melo Costa, F. Popineau, A. Rimmel, and B.-L. Doan, “T-jepa: Augmentation-free self-supervised learning for tabular data,” in *International Conference on Learning Representations (ICLR)*, 2025.
- [15] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “Lightgbm: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [16] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, “Catboost: unbiased boosting with categorical features,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [17] A. Benmalek, S. Ounacer, and S. Aridhi, “Ratab: Representation space augmentation for tabular data,” in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2025.
- [18] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo, “Cutmix: Regularization strategy to train strong classifiers with localizable features,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.
- [19] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, “mixup: Beyond empirical risk minimization,” in *International Conference on Learning Representations (ICLR)*, 2018.
- [20] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2019.